

Introduzione ai Sistemi Operativi

Un programma in esecuzione esegue istruzioni.

Il processore recupera un'istruzione dalla memoria

La decodifica

La esegue

Successivamente si sposta sulla prossima istruzione (Modello di Von Neumann).

Il sistema operativo è responsabile di rendere semplice l'esecuzione di programmi, permettere ai programmi di condividere la memoria, permettere ai programmi di interagire con gli altri dispositivi. Il sistema operativo ha il compito di controllare che il sistema lavori in modo efficiente e corretto. Il sistema operativo prende una risorsa fisica e la trasforma in una virtuale. Sostanzialmente grazie alle chiamate di sistema permettiamo all'utente di dire cosa fare al sistema operativo. Quindi un sistema operativo gestisce le risorse, condivisione della cpu e condivisione delle memorie. Il sistema operativo virtualizzando la cpu ci dà l'impressione che questa utilizzi un numero infinito di cpu (Guarda cpu.c). Per quanto riguarda la virtualizzazione della memoria la memoria viene letta: specificando un indirizzo dove accedere ai dati e viene scritta: specificando i dati da scrivere all'indirizzo evocato (Guarda mem.c). Nella virtualizzazione della memoria ogni processo accede al suo spazio di indirizzi virtuali privati. Il sistema operativo mappa lo spazio degli indirizzi sulla memoria fisica. La memoria fisica è una risorsa condivisa gestita dal sistema operativo. Un altro problema è quello della concorrenza dato dai programmi multi thread. Si può immaginare che il nostro programma segua un filo principale e che durante l'esecuzione di possano creare dei nuovi fili secondari (Guarda threads.c). Quando in un programma abbiamo il semplice incremento di una variabile, in realtà stiamo eseguendo tre istruzioni: caricare il valore della variabile dalla memoria al registro, incrementarla e rimetterla nella memoria; Queste tre istruzioni non vengono eseguite atomicamente, vengono infatti considerate non un'unica istruzione ma tre diverse. Persistenza, hardware e software devono essere in grado di contenere dati persistentemente (file, dati...), Il sistema operativo si occupa di capire dove andranno messi i nuovi dati, infatti emette delle richieste al dispositivo di archiviazione sottostante. I goal del sistema operativo sono quindi: costruire un'astrazione e quindi rendere il sistema semplice da utilizzare, fornire delle buone performance, protezione tra le applicazioni e essere affidabile.

Il processo di astrazione

Processo: applicazione in esecuzione (Un'applicazione è un insieme di istruzioni su un qualche dispositivo di archiviazione).

Il meccanismo che ci permette di eseguire più programmi con poche cpu è il time sharing, consiste nell'assegnare un'unità di tempo (quanto) a un processo, quando questa porzione di tempo finisce ho due opzioni 1. Finisco il processo che stavo eseguendo e passo a un altro 2. Non sono riuscito a finire il processo lo blocco momentaneamente per passare a un altro. Il meccanismo di stoppare un processo e passare a un altro viene detto context switching. Ovviamente il context switching è basato su alcune policy che prendono la decisione su quando stoppare un processo per quanto ecc... Tornando ai processi un processo è composto da la parte della memoria come istruzioni e dati e la parte dei registri come program counter e stack pointer. Il sistema operativo si occupa di controllare il ciclo di vita dei processi, lo fa con le seguenti funzioni: lo crea, lo distrugge, può metterlo in stato di wait, gli fa i vari controlli e ottiene informazioni sullo stato del processo. Riguardo invece alla creazione di un processo: viene caricato il codice di un programma nella memoria, nello spazio degli indirizzi di un processo, si dice che il sistema operativo carica i processi in maniera "pigra" (Per ottimizzare

l'utilizzo della memoria si carica il minimo indispensabile nella memoria e poi quando le funzioni vengono richieste viene allocata ulteriore memoria). Viene allocato lo stack a tempo di esecuzione, solitamente lo stack è utilizzato per variabili locali, parametri di funzioni e return address. Viene creato l'heap del programma, utilizzato per i dati allocati dinamicamente (malloc ()), vengono inizializzate le altre cose e successivamente viene fatto eseguire il programma. Un processo può avere tre stati: in esecuzione (running) , pronto (ready): il processo è pronto per andare in esecuzione ma per qualche motivo il sistema operativo ha deciso di non eseguirlo in questo momento, bloccato (blocked). Il sistema operativo ha delle strutture dati per organizzare le informazioni es. xv6 kernel proc structure. Ritornando al time sharing l'obiettivo del sistema operativo è quello di realizzare il context switching con meno overhead possibile, ovvero tutto quell'insieme di istruzioni che posso evitare di eseguire. Un altro obiettivo è quello di avere, da parte del SO, il controllo di quello che sta succedendo. Alcuni processi hanno bisogno di effettuare operazioni che non tutti potrebbero fare, per questo motivo nasce la problematica dell'implementare un meccanismo che gestisca i permessi di questi processi. La soluzione è stata creare due modelli di lavoro. Uno per i processi regolari che non hanno accesso a tutte le risorse hardware, detto User Mode e l'altro per dei processi particolari che hanno accesso a tutte le risorse detto Kernel Mode. Le chiamate di sistema fanno sì che i processi abbiano accesso alle funzionalità della kernel-mode. Per eseguire una chiamata di sistema un programma lancia un'istruzione trappola che salta nel kernel e aumenta il livello di priorità a kernel-mode. Dopo quest'istruzione trappola tutto ritorna come prima e il livello di privilegio ridiventa User Mode. Ovviamente dato che saltare da una modalità all'altra sarebbe un pericolo il Kernel ha una Trap table dove un numero è associato a ogni chiamata di sistema. Come fa il sistema operativo a riprendere il controllo della cpu, in modo da poter passare da un processo a un altro: ci sono due approcci, Uno cooperativo (aspettare per le chiamate di sistema) e uno non cooperativo (il sistema operativo prende il controllo). I processi cedono regolarmente la CPU effettuando chiamate di sistema come yield(). Le applicazioni trasferiscono il controllo al sistema operativo quando stanno facendo qualcosa di illegale (divisione per 0). L'utilizzo di questo approccio è spesso meno che ottimale: uno dei motivi potrebbe essere un processo cattivo che propositamente non fa nessuna chiamata di sistema. Per quanto riguarda l'altro approccio al sistema operativo serve che l'hardware sia collaborativo per realizzare un approccio non cooperativo. Timer interrupt: Durante la sequenza di avvio il sistema operativo inizializza un timer, il timer genera un'interruzione ogni tot tempo. Quando viene generata un'interruzione il processo in esecuzione viene arrestato, lo stato del programma viene salvato, un gestore di interruzioni preconfigurato nel sistema operativo viene eseguito, Questo Timer Interrupt consente al sistema operativo di riprendere l'esecuzione su una CPU. Quando si verifica un Timer Interrupt il sistema operativo si occupa di disabilitare le interruzioni date dal timer e si occupa di utilizzare degli schemi che impediscono l'accesso concorrente.